

Laboratory 02

FreeRTOS Basics: Tasks, Queues, Semaphores & Timers

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 96 MHz)
RTOS	FreeRTOS (bare-metal, no SDK)
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Tools	pyocd, serial terminal (115200 8N1)
Estimated time	2–3 hours
CLO mapping	CLO 3 (FreeRTOS implementation) CLO 4 (synchronisation)

1. Objectives

By completing this lab you will be able to:

1. Build a multi-task FreeRTOS application that uses four kernel primitives — task, queue, semaphore, and software timer.
2. Explain the **producer–consumer** pattern and observe what happens when a queue overflows.
3. Use a **mutex** to protect a shared resource and demonstrate the corruption that occurs without one.
4. Distinguish a **binary** semaphore from a **counting** semaphore by observing lost signals.
5. Configure a **software timer** (one-shot vs. auto-reload) and explain the rules for timer callbacks.
6. Contrast `vTaskDelay` and `vTaskDelayUntil` and measure timing drift.

2. Prerequisites

You must have completed **Lab 01** before starting:

- ☐ ARM GNU Toolchain installed and working (make succeeds in `lab01_setup_verify/`)
- ☐ pyocd installed (`pip install pyocd`)
- ☐ FRDM-MCXXN236 connected via the **MCU-LINK USB** port
- ☐ A serial terminal (`screen`, `minicom`, PuTTY, or Tera Term) at **115200 8N1**

💡 Tip

No SEGGER SystemView is required for this lab — all observation is done over UART.

3. Project Structure

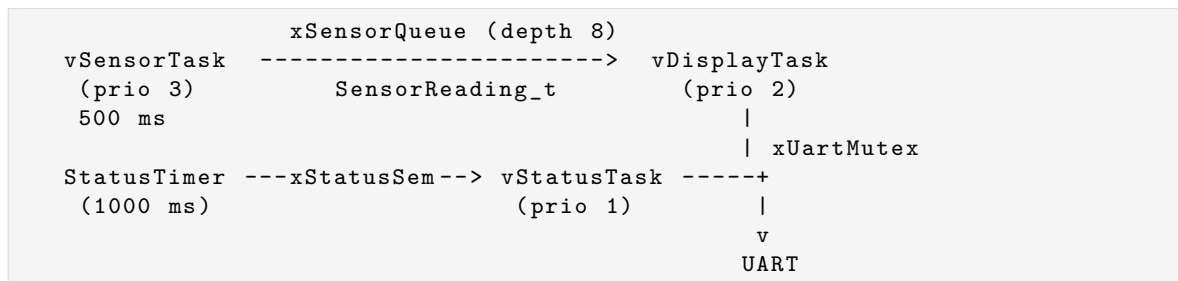
```
lab02_freertos_basics/
  Makefile                <- build, flash, and debug targets
  linker_MCXXN236.ld      <- flash/RAM memory map
  src/
    startup_MCXXN236.s    <- vector table + Reset_Handler
    main.c                <- hw_init, RTOS objects, scheduler start
    lab_tasks.c / .h      <- the three tasks + timer callback
    FreeRTOSConfig.h      <- kernel configuration
    uart.h / uart.c       <- bare-metal LPUART4 driver
    FreeRTOS-Kernel/      <- (cloned by 'make setup')
```

3.1. What the code does — the “Sensor Data Pipeline”

This lab simulates a small sensor pipeline. Three tasks and one software timer cooperate through shared kernel objects:

Task / Object	Priority	Trigger	What it does
vSensorTask	3 (HIGH)	500 ms period (vTaskDelayUntil)	Generates a fake ADC reading, sends it to xSensorQueue
vDisplayTask	2 (MED)	Blocks on xSensorQueue	Receives a reading and prints it (under xUartMutex)
vStatusTask	1 (LOW)	Blocks on xStatusSem	Prints uptime + queue depth (under xUartMutex)
StatusTimer	— (timer task)	1000 ms, auto-reload	Callback gives xStatusSem to wake vStatusTask

Data flow:



- **Queue** (xSensorQueue) decouples the producer (Sensor) from the consumer (Display).
- **Semaphore** (xStatusSem) is a pure *signal* — the timer signals, the task waits.
- **Mutex** (xUartMutex) guarantees only one task drives the UART at a time.

4. Part A — Build and Flash

4.1. Step 1 — Clone the FreeRTOS kernel

```
cd labs/lab02_freertos_basics
make setup
```

This runs `git clone -depth=1` into `FreeRTOS-Kernel/`. You only do this once.

💡 Tip

Unlike Lab 01, this lab’s Makefile also compiles `timers.c` — the software-timer module — because we use a software timer.

4.2. Step 2 — Build

```
make
```

Expected output (sizes will vary slightly):

```
arm-none-eabi-gcc ... -o lab02.elf
arm-none-eabi-objcopy -O binary lab02.elf lab02.bin
text      data      bss      dec      hex filename
1xxxx     xxx      xxxxxx  xxxxxx  xxxx lab02.elf
```

 Note

If `arm-none-eabi-gcc`: not found, set the toolchain path at the top of Makefile:

```
TOOLCHAIN ?= /path/to/arm-none-eabi/bin
```

4.3. Step 3 — Flash

```
make flash
```

pyocd erases, programs, and resets the MCU. The board runs immediately.

5. Part B — UART Output Verification

Open a serial terminal:

```
ls /dev/tty.usbmodem*          # find the port
screen /dev/tty.usbmodemXXXX 115200
```

Press the board **RESET** button to see the banner from the start.

5.1. Expected output

```
=== RTOS Lab 02: FreeRTOS Basics ===
Tasks: Sensor | Display | Status
Primitives: Queue | Mutex | Semaphore | Timer

[MAIN] Starting scheduler...

[DISPLAY] t=0 ms   value=7
[DISPLAY] t=500 ms value=14

[STATUS] uptime=1000 ms queue_waiting=0
[DISPLAY] t=1000 ms value=21
[DISPLAY] t=1500 ms value=28

[STATUS] uptime=2000 ms queue_waiting=0
```

 Tip**Key observations:**

- Two [DISPLAY] lines appear for every one [STATUS] line — 500 ms vs. 1000 ms.
- `queue_waiting=0` — the consumer keeps up with the producer, so the queue stays empty.
- The `value` field steps by 7 each reading (`raw = (raw + 7) % 101`).
- Lines never interleave — the mutex serialises UART access.

 Checkpoint A

Show a terminal capture of at least 5 seconds of correct output to your instructor before continuing.

6. Part C — Experiments

Work through these in order. For each experiment: make the edit, run `make flash`, observe the UART, then answer the question.

 Note

Restore the original code after each experiment unless told otherwise.

6.1. Experiment 1 — The queue as a producer–consumer buffer

The producer (Sensor) runs every 500 ms. The consumer (Display) normally keeps up. Now make the consumer **slower than** the producer.

In `lab_tasks.c`, inside `vDisplayTask`, add a delay after the print:

```
1 xSemaphoreGive(xUartMutex);
2 vTaskDelay(pdMS_TO_TICKS(900)); /* <-- ADD: slow consumer */
```

Rebuild, flash, and watch the `queue_waiting` count in the [STATUS] lines.

? Question 1

- The producer makes 2 readings/s; the consumer now drains $\sim 1.1/s$. Watch `queue_waiting` climb. The queue depth is 8 (see `xQueueCreate` in `main.c`) — roughly how many seconds until you see [SENSOR] Queue full -- reading dropped!?
- `xQueueSend` in `vSensorTask` uses a timeout of 0. What would change if it used `portMAX_DELAY` instead? Would readings still be dropped?
- In one sentence: what problem does a queue solve that a shared global variable does not?

Restore `vDisplayTask` (remove the added `vTaskDelay`).

6.2. Experiment 2 — The mutex and the race condition

`xUartMutex` ensures `Display` and `Status` never write the UART at the same time. Remove it and force a collision.

- In `lab_tasks.c`, comment out **all four** `xSemaphoreTake(xUartMutex, ...)` and `xSemaphoreGive(xUartMutex)` calls in `vDisplayTask` and `vStatusTask` (keep the `uart_printf` lines).
- To make collisions frequent, speed both producers up:
 - `vSensorTask`: change `xPeriod` to `pdMS_TO_TICKS(50)`.
 - `main.c`: change the timer period to `pdMS_TO_TICKS(100)`.

Rebuild, flash, and study the terminal.

? Question 2

- Do you see garbled lines — a [DISPLAY] fragment spliced into the middle of a [STATUS] line, or vice versa?
- This is a **race condition**. Explain why it happens *even on a single-core MCU*. (Hint: the UART sends one character at a time; the scheduler can preempt `Display` mid-string.)
- FreeRTOS mutexes implement **priority inheritance** (`configUSE_MUTEXES 1`), but binary semaphores do not. Why does that matter when a low-priority task holds a lock that a high-priority task wants?

Restore the mutex calls and the original periods.

6.3. Experiment 3 — Binary vs. counting semaphore (lost signals)

`xStatusSem` is a **binary** semaphore: it holds at most one “token”. If the timer gives it again before `vStatusTask` takes it, the extra give is **discarded**.

- In `main.c`, speed up the timer: `pdMS_TO_TICKS(1000) → pdMS_TO_TICKS(200)`.
- In `lab_tasks.c`, slow down `vStatusTask` by adding a delay after the print:

```
1 xSemaphoreGive(xUartMutex);
2 vTaskDelay(pdMS_TO_TICKS(1000)); /* <-- ADD */
```

Rebuild and flash. The timer now fires $\sim 5 \times /s$, but `vStatusTask` can only process $\sim 1 \times /s$.

? Question 3a

How many [STATUS] lines appear per second? How many timer “gives” are being **lost**?

Now switch to a **counting** semaphore. In `main.c`:

```
1 xStatusSem = xSemaphoreCreateCounting(10, 0); /* was
   xSemaphoreCreateBinary() */
```

(`configUSE_COUNTING_SEMAPHORES` is already 1 in `FreeRTOSConfig.h`.) Rebuild and flash.

? Question 3b

With the counting semaphore, what happens when `vStatusTask` finally runs after its 1000 ms delay? Why does it now print several lines back-to-back? When is a binary semaphore the *correct* choice, and when do you need a counting one?

Restore the binary semaphore, the 1000 ms timer period, and `vStatusTask`.

6.4. Experiment 4 — Software timer: one-shot vs. auto-reload

The `StatusTimer` is created with `pdTRUE` — **auto-reload**. Change it to one-shot. In `main.c`, in the `xTimerCreate` call:

```
1 xTimerCreate("StatusTimer", pdMS_TO_TICKS(1000),
2             pdFALSE, /* <-- was pdTRUE: now one-shot */
3             NULL, vStatusTimerCallback);
```

Rebuild and flash.

? Question 4

- How many [STATUS] lines do you see now, and why?
- The callback `vStatusTimerCallback` runs in the **timer service task** (`configTIMER_TASK_PRIORITY = 4`, the highest priority). Why must a timer callback **never** call a blocking API such as `vTaskDelay` or a `Take` with a long timeout?
- Name one way to make a one-shot timer fire repeatedly *without* using `pdTRUE`. (Hint: `xTimerStart` / `xTimerReset` can be called from inside the callback.)

Restore `pdTRUE`.

6.5. Challenge (optional) — vTaskDelay vs. vTaskDelayUntil

`vSensorTask` uses `vTaskDelayUntil` for a **drift-free** 500 ms period. Replace it with `vTaskDelay` and add a variable workload:

```
1 /* Simulate variable processing time before the delay */
2 for (volatile int i = 0; i < 200000; i++) { }
3
4 vTaskDelay(pdMS_TO_TICKS(500)); /* <-- was vTaskDelayUntil
   (...) */
```

Rebuild, flash, and look at the `t=` timestamps in the [DISPLAY] lines.

? Question Challenge

With `vTaskDelay`, the period becomes *work time + 500 ms*, so the timestamps drift away from clean multiples of 500. With `vTaskDelayUntil` they stay locked to the 500 ms grid. Explain why — what does `vTaskDelayUntil` measure its delay *from* that `vTaskDelay` does not?

Restore the original `vTaskDelayUntil` code.

7. Deliverables

Submit the following to the course LMS by the posted deadline:

#	Item	Details
1	Baseline capture	5+ seconds of correct output (Checkpoint A)
2	Experiment 1 capture	Terminal output showing a Queue full drop
3	Experiment 2 capture	Terminal output showing garbled (interleaved) lines
4	Experiment 3 captures	Before <i>and</i> after the counting semaphore
5	Answers to Q1–Q4	A few sentences each; reference the code or output
6	Challenge answer	Optional, if attempted

8. Key Concepts Introduced

Concept	Where you saw it
Producer–consumer decoupling via a queue	Experiment 1
Queue overflow and non-blocking <code>xQueueSend</code>	Experiment 1
Race condition on a shared resource	Experiment 2
Mutex as a critical-section guard; priority inheritance	Experiment 2
Binary vs. counting semaphore; lost signals	Experiment 3
Software timers: one-shot vs. auto-reload; callback rules	Experiment 4
Drift-free periodic timing	Challenge

These primitives are the toolkit for Module 3 (synchronisation) and Module 4 (inter-task communication).

9. Reference

Resource	Relevant sections
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 4 (queues), Ch. 6 (semaphores/mutexes), Ch. 5 (software timers)
Laplante, <i>Real-Time Systems Design</i>	Ch. 3 (intertask communication, synchronisation)
FreeRTOS documentation	xQueueCreate, xQueueSend, xQueueReceive, xSemaphoreCreateMutex, xSemaphoreCreateBinary, xSemaphoreCreateCounting, xTimerCreate
